Chốt lại đội hình bằng lăng kính nguyên lý cơ bản (First Principles): Nếu 7 kỹ sư trước đã xây dựng một cỗ máy định tuyến không gian, thời gian và toán học hoàn hảo, nhưng khách du lịch không thể nhìn thấy và tương tác với nó, thì dự án này trị giá 0 đồng.

Dev 8 (UI/UX) chính là **Bề mặt Hiển thị (The Surface)** của toàn bộ hệ thống.

[FACT] Trạng thái hiện tại của UI: Hoàn toàn là dựng cảnh (Mockup). Code trong `mobile/App.tsx` chỉ là các nút bấm thủ công và dòng chữ thú nhận: *"Replace this card with a geofenced map component..."*.

[CRITICAL RISK] Giao diện hiện tại không thể hiện được bất kỳ dữ liệu thực nào từ Backend. Nếu Dev 8 tiếp tục sa đà vào việc "làm UI cho đẹp" (animation, màu sắc) mà bỏ qua tính kết nối I/O thực tế, dự án sẽ thất bại ở khâu demo cuối cùng.

Mệnh lệnh thực thi hardcore dành cho Dev 8:

1. Xóa bỏ Ảo giác (Kill the Web-view & Mockups)

- **Nhiệm vụ:** Tiêu diệt toàn bộ các thành phần giả lập trong `App.tsx`.
- **Hành động bắt buộc:** Theo đúng thiết kế, Dev 8 phải loại bỏ tư duy làm web-view tĩnh và tích hợp trực tiếp **React Native Maps** (Map SDK) vào ứng dụng. Khu vực màn hình chính phải là một bản đồ tương tác được, không phải một tấm ảnh hay thẻ (card) tĩnh.

2. Trực quan hóa Toán học (Render the Polyline)

- **Phân tích I/O:** Backend của Dev 4 và Dev 5 sẽ nhả về một cấu trúc dữ liệu JSON chứa tọa độ của 10 điểm đến (Node) theo thứ tự tối ưu.
- **Hành động bắt buộc:** Dev 8 phải viết hàm parse dữ liệu JSON này và vẽ một đường Polyline (hành trình) nối chính xác các tọa độ đó trên bản đồ React Native Maps. Nếu OR-Tools bảo khách đi từ Đại Nội sang Chùa Thiên Mụ, UI phải vẽ đúng một đường nét đứt (hoặc nét liền) nối hai tọa độ vật lý đó trên màn hình.

3. Đồng bộ Trạng thái Động (Timeline & Reroute UI)

- **[GAP] Lỗ hổng:** Khi người dùng bị trễ và Dev 7 (Geofencing) bắn event ngầm lên server để tính lại đường, làm sao người dùng biết lịch trình của họ đã thay đổi?
- **Hành động bắt buộc:** * **UI Timeline:** Xây dựng một giao diện dạng trục thời gian (Timeline View) hiển thị danh sách các điểm đến cùng Giờ dự kiến (ETA).
    - **Reroute Handling:** Khi nhận được tín hiệu Notification (FCM) từ Dev 6 trả về máy, giao diện của Dev 8 phải tự động re-render (vẽ lại) bản đồ và timeline sang lộ trình mới. Phải có một pop-up hoặc dải băng cảnh báo rõ ràng (Ví dụ: "Đã cập nhật lại lịch trình do bạn trễ 30 phút").

Chỉ đạo nghiệm thu (Acceptance Criteria) cho Dev 8:

Không cần UI bóng bẩy. Cách nghiệm thu: Nhập prompt "Tôi muốn đi 3 lăng tẩm". Bấm Generate. UI phải vẽ ra đúng 3 điểm trên bản đồ kèm đường nối. Sau đó, dùng Postman (đóng vai backend) bắn một payload Reroute chứa 3 điểm hoàn toàn khác (do thời tiết mưa)

xuống app. Nếu màn hình App tự động chớp và vẽ lại thành đường mới mà không bị văng (crash), Dev 8 hoàn thành xuất sắc nhiệm vụ.

Với tư duy nguyên lý cơ bản (First Principles), nếu Dev 1 lo "vật lý không gian", Dev 2 lo "động cơ", Dev 3 lo "phiên dịch", thì **Dev 4 (Maths)** chính là người chế tạo **Bộ Não (The Brain)** của dự án này.

Toán học không biết nói dối. Mã nguồn hiện tại trong file backend/app/routing_solver.py được đánh giá là phần AI đang làm tốt nhất. Dev 4 đã viết sẵn cơ chế Fallback (chống sập) rất thông minh: từ OR-Tools lùi về Tham lam (Greedy), lùi về Xếp hàng (Sequential).

Tuy nhiên, bộ não này hiện tại đang bị "nhốt trong lồng kính" và chơi với dữ liệu ảo. Nhiệm vụ của Dev 4 trong Sprint này là cắm bộ não đó vào thế giới thực. Dưới đây là các đầu ra (Deliverables) mang tính sinh tử mà Dev 4 phải thực thi:

1. Nối Ống Dữ Liệu (The I/O Pipeline)

- **Tình trạng hiện tại:** Bộ não chưa được kết nối. Hệ thống có nguy cơ sập do "Combinatorial Explosion" (bùng nổ tổ hợp) nếu ném nguyên mảng 1000 điểm JSON vào OR-Tools.
- **Hành động bắt buộc:** Dev 4 phải dẹp bỏ logic đọc mảng JSON chay. Phải viết hàm map (ánh xạ) trực tiếp đầu ra của PostGIS (sau khi đã được Lớp 3 lọc xuống còn đúng 50 điểm) vào đầu vào của routing_solver.py.
- **Kiểm chứng logic:** Thuật toán chỉ được phép giải bài toán nội trong ma trận 50x50, nhả ra tối đa 10 điểm đến tối ưu nhất. Vượt quá số này, latency sẽ thủng mức 3 giây.

2. Quản lý Ràng buộc Vật lý (Hard/Soft Constraints Constraints)

- **Phân tích:** OR-Tools không hiểu thế nào là "vui" hay "buồn", nó chỉ hiểu Hàm Mục Tiêu (Objective Function).
- **Hành động bắt buộc:**
    - **Hard Constraints (Bất khả xâm phạm):** Dev 4 phải đảm bảo biến open_time và close_time từ DB được truyền đúng vào tham số Time Windows của OR-Tools. Tuyệt đối không được xếp lịch trình để khách đến Đại Nội lúc 18h00 khi cửa đã khóa.
    - **Soft Constraints (Hàm Phạt - Penalty):** Tính năng né thời tiết hiện đã được code cứng (nếu có biến rain + outdoor thì đẩy weather_penalty lên 999). Dev 4 cần review lại code này để đảm bảo khi Dev 3 truyền biến thời tiết vào, thuật toán sẽ tự động bẻ lái lộ trình sang các điểm Indoor (trong nhà) mà không làm sập (crash) hệ thống.

3. Khai hỏa Live Traffic bằng Just-In-Time (JIT)

- **[GAP] Lỗ hổng:** Hiện tại hệ thống định tuyến hoàn toàn phụ thuộc vào khoảng cách tĩnh hoặc đường chim bay.
- **Hành động bắt buộc (Sniper API):** Dev 4 không được phép gọi API Google Maps Live Traffic cho toàn bộ ma trận (hành động đốt tiền). Hard requirement ở đây là phải cấu hình hệ thống theo mô hình Just-In-Time: **Chỉ gọi API cho đúng 1 điểm đến tiếp theo**.
- **Kịch bản thực thi:** Khi người dùng đang đứng ở Điểm A, chuẩn bị đi Điểm B. Hệ thống của Dev 4 sẽ "bắn tỉa" 1 request duy nhất lên Google Maps. Nếu Google trả về

duration_in_traffic chênh lệch quá 30 phút so với OSRM do kẹt xe, Dev 4 lập tức gửi tín hiệu Re-route cho Dev 6.

Nghiệm thu (Acceptance Criteria) cho Dev 4: Output của bạn là lõi routing_solver.py. Chạy file test_routing.py bằng lệnh unit test. Hệ thống phải nuốt 50 tọa độ thực, query ma trận OSRM của Dev 2, ép qua Time Windows, và xuất ra 1 mảng JSON chứa 10 điểm theo thứ tự A $\rightarrow$ B $\rightarrow$ C có thời gian đến (ETA) hợp lý. Mọi thứ phải xảy ra dưới 3 giây. Không bao biện.

Dưới góc nhìn thiết kế hệ thống (First Principles), Dev 3 (LLM Ops) đang nắm giữ vị trí "Người Gác Cổng" (The Gateway) giữa hai thế giới: sự hỗn loạn của ngôn ngữ tự nhiên (con người) và sự chặt chẽ tuyệt đối của toán học (thuật toán OR-Tools).

Bản chất của các mô hình ngôn ngữ lớn (LLM) là các cỗ máy dự đoán từ theo xác suất. Chúng sinh ra văn bản rất hay, nhưng lại cực kỳ dốt trong việc duy trì cấu trúc cú pháp 100% thời gian. Nếu chỉ dùng "prompt chay", LLM sẽ nhả ra JSON thiếu dấu phẩy, thừa text giải thích, và ngay lập tức đánh sập toàn bộ hệ thống định tuyến phía sau.

Nhiệm vụ của Dev 3 không phải là "ngồi chat với AI". Nhiệm vụ của Dev 3 là xây dựng một cái gông cùm bằng code để ép AI tuân thủ luật chơi.

1. Xây dựng Lớp Ép Kiểu (Structured Outputs) - Không Khoan Nhượng

- **[CRITICAL RISK]:** Trong tài liệu thiết kế đã cảnh báo rõ mức độ nguy hiểm: Nếu prompt thô truyền vào không ra JSON chuẩn, hệ thống định tuyến phía sau (Lớp 4) bị liệt hoàn toàn.
- **Hành động bắt buộc:** Dev 3 phải đi thẳng vào file `backend/app/parser.py`, cài đặt và tích hợp thư viện **Instructor** kết hợp cùng **Pydantic**.
- **Cơ chế hoạt động:** Bất kể người dùng gõ câu lệnh kỳ quặc đến đâu (VD: "Tui muốn đi lượn lờ và ăn đồ cay, chiều phải về lúc 5h, sinh viên nghèo"), parser này không được phép trả lời bằng văn bản. Nó phải ép OpenAI/Gemini nhả ra chính xác định dạng JSON chuẩn 100%.
- **Output mẫu kỳ vọng:** `{"budget_max": 2000000, "soft_tags": ["culture", "spicy"], "hard_start": "08:00"}`.

2. Thiết kế Prompt System (Bộ não định tuyến)

- **Tình trạng:** Hiện tại code chứa `parser.py` nhưng chưa thấy nội dung implement Pydantic/Instructor đúng như thiết kế.
- **Hành động:** Dev 3 phải thiết kế một System Prompt sắc bén. Prompt này phải có khả năng đọc hiểu ý định của người dùng và chuyển đổi (map) thành các biến số cụ thể:
    - Chuyển hóa sở thích thành `soft_tags` (để khớp với dữ liệu POI mà Dev 1 cào về).
    - Trích xuất các ràng buộc thời gian cứng (Hard constraints) để nhồi vào tham số Time Windows cho Dev 4 xử lý.

3. Thiết lập Mạng lưới An toàn (Unit Test Chống Sập & Fallback)

- **Phân tích logic:** LLM là API bên thứ 3. Nó có thể timeout, nó có thể từ chối trả lời do bộ lọc an toàn, hoặc nó bị ảo giác (hallucination). Hệ thống của chúng ta không được phép sụp đổ vì API của OpenAI/Google gặp sự cố.
- **Hành động:** * Dev 3 phải viết các bài Unit Test (kiểm thử) tự động. Đặt mã kiểm thử vào thư mục `backend/tests/`.
    - Phải ném các đầu vào rác (Garbage inputs) vào Parser để thử đánh sập nó.
    - Đọc hiểu và xây dựng cơ chế Fallback (Lùi bước). Nếu LLM thất bại trong việc trả về JSON chuẩn sau 2-3 lần thử (retries), hàm parser phải tự động

nhả ra một cấu hình JSON an toàn mặc định (Default Parameters) để luồng chạy `parse -> filter -> matrix -> solve` trong file `planner.py` không bị đứt gãy.

Chỉ đạo nghiệm thu (Acceptance Criteria) cho Dev 3:

Output của bạn không phải là một chuỗi chat dài dòng. Vật phẩm bàn giao là hàm API POST `/plan`. Khi người dùng bấm nút Submit trên điện thoại, trong vòng chưa tới 1 giây, file `parser.py` của bạn phải bắt dính input, ném cho LLM, khóa chặt định dạng đầu ra, và ném một cục JSON không sai một dấu phẩy sang cho Dev 4. Tỷ lệ lỗi parsing (Parsing Error Rate) phải là 0%. Làm ngay đi.

Tiếp tục giải phẫu hệ thống bằng tư duy nguyên lý cơ bản (First Principles): Nếu Dev 4 là "Bộ não", Dev 6 là "Hệ thần kinh", thì **Dev 7 (Map & Geo)** chính là **Cảm biến vật lý (The Physical Sensor)** của dự án này.

Toán học có tối ưu đến đâu mà không có dữ liệu thực tế cập nhật thì cũng chỉ là thứ đồ chơi trong phòng thí nghiệm.

[FACT] Trạng thái hiện tại của Lớp 1 (Mobile App): Hoàn toàn là một cú lừa (Mockup). Giao diện hiện tại chỉ có một cái nút bấm tay "I'm delayed" và lời thú nhận rành rành trong code App.tsx: *"Replace this card with a geofenced map component..."*.

[CRITICAL RISK] Không một ai đang đi du lịch lại muốn cứ 15 phút phải rút điện thoại ra bấm nút thông báo cho app là "tôi đang ở đây". Nếu Dev 7 thất bại, toàn bộ ý tưởng "Tự động Re-route" của dự án sẽ chết yểu ngay từ vạch xuất phát.

Dưới đây là mệnh lệnh thực thi hardcore dành cho Dev 7:

1. Đập bỏ "Hộp cát" (Kill Expo Go)

- **[GAP] Vấn đề:** App hiện tại đang chạy qua ứng dụng Expo Go (thể hiện qua file expo-go-qr.png). Nhưng Expo Go là một môi trường bị hạn chế (Sandbox), nó KHÔNG hỗ trợ chạy ngầm (background location) một cách tử tế, hệ điều hành sẽ giết chết tiến trình ngay khi màn hình tắt.
- **Hành động bắt buộc:** Dev 7 phải vứt bỏ Expo Go ngay lập tức. Hard Requirement là phải cấu hình và build ra một bản **Expo Development Build (Custom Dev Client)** thực sự. Chỉ có ứng dụng Native mới có thể xin quyền OS (iOS/Android) để tracking vị trí chạy ngầm "Always Allow".

2. Cắm Hàng rào ảo (The 500m Geofencing Trap)

- **Phân tích nguyên lý:** Tại sao thiết kế lại cấm "Live GPS" mà bắt dùng "Geofencing"?. Vì nếu ping GPS liên tục mỗi giây để vẽ đường như Google Maps, pin điện thoại sẽ sập trong 2 tiếng đồng hồ.
- **Hành động bắt buộc:** Dev 7 phải tích hợp một thư viện Geofencing Native (như expo-location với tính năng geofencing).
    - Khi người dùng (khách du lịch) bước vào Điểm A (VD: Chùa Thiên Mụ), Dev 7 sẽ cắm một cột mốc ảo bán kính **500m** xung quanh tọa độ đó.
    - Thuật toán: Ngừng ping GPS liên tục. Giao toàn bộ việc canh gác cái vòng tròn 500m này cho hệ điều hành (OS). OS quản lý pin tốt hơn app rất nhiều.

3. Logic Đánh thức và Bắn Sự kiện (The Event Trigger)

- **Nhiệm vụ:** App phải bị OS đánh thức (wake up) ngay khoảnh khắc người dùng bước ra khỏi vùng 500m.
- **Điều kiện kích hoạt (Trigger Condition):** Không phải cứ ra khỏi vùng là bắn API. Dev 7 phải viết code để so sánh:
    - Giờ dự kiến rời đi (Lấy từ state của Dev 5).
    - Giờ thực tế bước ra khỏi Geofence.
    - [INFERENCE] Nếu user ra khỏi bán kính 500m mà **trễ hơn dự kiến** (ví dụ: lịch ghi 15:00 ra mà 15:30 mới ra), lúc này đoạn code của Dev 7 mới thu thập

tọa độ, đóng gói thành file JSON (theo đúng format `contracts/reroute-request.schema.json`) và bắn tín hiệu (Event) lên backend cho Dev 6 xử lý.

Chỉ đạo nghiệm thu (Acceptance Criteria) cho Dev 7:

Nghiệm thu không được làm trên Emulator (máy ảo). Dev 7 phải cầm một chiếc điện thoại thật, cài bản build Native. Khóa màn hình, nhét điện thoại vào túi quần. Cầm xe chạy ra khỏi nhà vượt quá 500m. Mở log server lên: Nếu thấy bắn lên 1 request API `/reroute` thành công trong khi màn hình điện thoại vẫn đang đen xì, thì tính năng Cảm Biến mới chính thức hoàn thành. Ngược lại, code làm lại từ đầu.

Dưới góc độ kiến trúc hệ thống, nếu Dev 1 là người rải "bản đồ vật lý" (tọa độ không gian), thì **Dev 2 (Infra)** chính là người xây dựng "động cơ vật lý" cho dự án này.

Mục tiêu sinh tử (Hard Requirement) của Dev 2 được định nghĩa rất rõ ràng: **"Tối ưu docker-compose cho OSRM Matrix, đảm bảo ping nội bộ < 50ms"**.

Tại sao con số 50ms lại quan trọng? Vì để OR-Tools tính toán đường đi qua 50 điểm, nó cần một Ma trận khoảng cách/thời gian (Distance/Time Matrix) kích thước 50x50 = 2500 cặp điểm. Nếu gọi API của Google Maps cho 2500 lượt, hệ thống sẽ sập về cả thời gian chờ (latency) lẫn chi phí tài chính. Do đó, Dev 2 bắt buộc phải dựng một máy chủ định tuyến Local (OSRM) hoàn toàn miễn phí và đạt tốc độ ánh sáng.

Dưới đây là bản "giải phẫu" chi tiết các công việc Dev 2 bắt buộc phải thực thi ngay lập tức:

1. Trám Lỗ hổng Dữ liệu Bản đồ (The Graph Data Gap)

- **Tình trạng hiện tại:** File `docker-compose.yml` đang gọi lệnh `osrm-routed --algorithm mld /data/vietnam-latest.osrm`. Tuy nhiên, theo tài liệu `README.md`, file dữ liệu `vietnam-latest.osrm` này **không hề tồn tại** trên repository (do dung lượng quá lớn) và hệ thống backend đang phải "lùi về" (fallback) dùng ma trận hình học (đường chim bay).
- **Hành động bắt buộc của Dev 2:** * **Không được để hệ thống chạy fallback đường chim bay, vì đường chim bay sẽ xuyên qua sông Hương và xuyên qua tường thành Đại Nội.**
    - Dev 2 phải viết script tải file PBF của Việt Nam (từ Geofabrik).
    - Chạy các container OSRM backend (osrm-extract, osrm-partition, osrm-customize) để tiền xử lý (pre-process) file PBF thành graph `vietnam-latest.osrm`.
    - Đặt đúng vào thư mục `osrm-data/` để container OSRM có thể mount volume lên và chạy.

2. Tối ưu Động cơ Định tuyến (OSRM Tuning)

- **Phân tích logic:** Trong `docker-compose.yml`, OSRM đang được khởi động với cờ `--algorithm mld` (Multi-Level Dijkstra).
- **Chiến lược của Dev 2:** * Thuật toán MLD ưu tiên tốc độ cập nhật dữ liệu giao thông động (Live traffic updates), nhưng trong phạm vi MVP này, chúng ta chưa có pipeline cập nhật kẹt xe real-time vào OSRM.
    - Do đó, Dev 2 cần đánh giá việc chuyển sang dùng `--algorithm ch` (Contraction Hierarchies). Thuật toán CH mất nhiều thời gian build (tiền xử lý) hơn, nhưng tốc độ nhả kết quả truy vấn Matrix (nhất là ma trận lớn 50x50) sẽ nhanh hơn rất nhiều so với MLD. Phải thử nghiệm cả hai nghiệm này để đảm bảo bảo vệ được mốc **< 50ms**.
    - Dev 2 cũng phải cấp đủ RAM (Memory limit) cho container OSRM trong `docker-compose.yml`, vì load bản đồ cả Việt Nam lên RAM cục bộ sẽ tiêu tốn khoảng vài Gigabytes. Nếu thiếu RAM, container sẽ crash (OOMKilled).

3. Đảm bảo Network nội bộ (Zero Network Overhead)

- **Phân tích logic:** API Backend giao tiếp với OSRM thông qua mạng nội bộ của Docker qua địa chỉ http://osrm:5000.
- **Hành động:** Dev 2 phải đảm bảo Docker network được cấu hình dạng bridge chuẩn. Mọi giao tiếp giữa API (Lớp 2) và OSRM (Lớp 4) không được đi vòng qua host network để tránh thất thoát tốc độ. Khi ping từ container itinerary-api sang itinerary-osrm, kết quả phải về ngay tức khắc.

4. Dựng các Service Phụ trợ (Redis & PostGIS)

- **Nhiệm vụ đi kèm:** Dev 2 cũng là người chịu trách nhiệm cho các file image của PostGIS (để Dev 1 nạp tọa độ) và Redis (để Dev 5 lưu state chuyến đi).
- **Hành động:** Đảm bảo container itinerary-postgis có mount volume postgis-data đàng hoàng để dữ liệu 500 điểm mà Dev 1 cào về không bị bốc hơi mỗi khi gõ lệnh docker compose down.

Kết luận cho Dev 2: Output nghiệm thu của Dev 2 không phải là code đẹp. Output của Dev 2 là một con số: **Bật Postman, bắn 1 request chứa 50 tọa độ vào http://localhost:5000/table/v1/driving/... và thời gian phản hồi (Response Time) trả về phải nhỏ hơn 50ms.** Nếu không đạt con số này, toàn bộ mục tiêu "Hệ thống sinh lịch trình < 3 giây" của dự án sẽ đổ sông đổ biển. Đứng dậy và đi build map ngay.

Hạng mục của Dev 2	Hành động thực thi cực đoan (Hardcore Execution)
1. Bơm dữ liệu bản đồ thật	Tải file bản đồ Việt Nam (dạng .pbf hoặc .osm.pbf). Chạy các script tiền xử lý của docker OSRM để build ra file vietnam-latest.osrm và đặt nó vào thư mục osrm-data/. Chấm dứt ngay trò tính đường chim bay.
2. Đổi thuật toán MLD sang CH	Mặc dù docker-compose.yml đang ghi là --algorithm mld, Dev 2 phải có tư duy phản biện. [INFERENCE] Thuật toán MLD tốt cho cập nhật kẹt xe real-time, nhưng hệ thống của chúng ta không đẩy kẹt xe vào OSRM. Do đó, Dev 2 nên đổi cờ khởi động sang thuật toán **CH (Contraction Hierarchies)** để tối đa hóa tốc độ truy xuất ma trận tĩnh.

3. Ép ping < 50ms	Đây là KPI sinh tử. Dev 2 phải config network của Docker để khi API Backend (Python) gọi qua container OSRM lấy ma trận khoảng cách cho 50 điểm tọa độ, tốc độ phản hồi không được vượt quá 50 mili-giây. Nếu chậm hơn, thuật toán tối ưu của Dev 4 sẽ bị nghẽn cổ chai.
4. Bảo vệ dữ liệu cho Dev 1	Container PostGIS hiện tại đã có Volume postgis-data. Dev 2 phải đảm bảo volume này mount chuẩn, để 500 điểm dữ liệu quý giá mà Dev 1 cào về bằng tay không bị xóa sạch mỗi khi gõ lệnh tắt server (docker compose down).

Nếu Dev 4 là người chế tạo **Bộ Não (The Brain)**, thì **Dev 5 (State/API)** chính là người xây dựng **Hệ Thần Kinh & Trí Nhớ (The Nervous System & Memory)**.

Hãy suy nghĩ theo nguyên lý cơ bản (First Principles): Hệ thống của chúng ta là một hệ thống "động" (Dynamic Routing). Khi người dùng báo "Tôi đến trễ" (Reroute), hệ thống không thể tính lại từ đầu nếu nó không nhớ người dùng đó *đang ở đâu* và *lịch trình cũ là gì*.

[FACT] Tình trạng hiện tại: Code trong file backend/app/state_store.py đang chạy bằng RAM (in-memory) để tiện cho việc test local.

[CRITICAL RISK] Chạy state trên RAM là tư duy của một dự án bài tập lớn, không phải của một sản phẩm thương mại. Nếu server khởi động lại, hoặc chúng ta scale (mở rộng) lên 2-3 containers, toàn bộ dữ liệu chuyến đi của khách du lịch sẽ bốc hơi ngay lập tức. Cấm tuyệt đối việc để code này lên production.

Dưới đây là mệnh lệnh thực thi hardcore dành cho Dev 5:

1. Dập tắt "Ảo giác bộ nhớ" (Kill the RAM State)

- **Nhiệm vụ:** Đi thẳng vào file backend/app/state_store.py. Xóa bỏ hoặc vô hiệu hóa đoạn code cho phép lưu trạng thái bằng bộ nhớ cục bộ.
- **Hành động bắt buộc:** Ép hệ thống đọc và ghi 100% vào service Redis đang chạy trong docker-compose.yml thông qua biến môi trường REDIS_URL=redis://redis:6379/0.

2. Quản lý State Chuyến Đi (Trip State Persistence)

- **Mục tiêu:** Lưu trữ chính xác những gì Dev 4 nhả ra.
- **Hành động:** Khi API /plan chạy xong, Dev 5 phải lấy mảng 10 điểm (Node) đó, gắn với một ID người dùng (User ID / Trip ID), và tống nó vào Redis. Phải lưu trữ rõ ràng:
    - Vị trí hiện tại (Current Node).
    - Điểm đến tiếp theo (Next Node).
    - Thời gian dự kiến (ETA) của từng chặng.

3. Xây dựng "Hệ thống Phanh" (Reroute Cooldown)

- **[GAP] Phân tích:** Nếu một người dùng bực tức và bấm nút "Reroute" liên tục 10 lần trên Mobile, hệ thống sẽ gọi 10 lệnh toán học phức tạp vào lõi OR-Tools, gây quá tải cục bộ.
- **Hành động:** Dev 5 phải thiết lập cơ chế khóa (Cooldown/Rate Limiting) trên Redis dựa vào biến REROUTE_COOLDOWN_SECONDS: 180 (3 phút) được định nghĩa trong file docker. Nếu user vừa reroute 1 phút trước, request tiếp theo phải bị API dội ngược ngay lập tức với mã lỗi 429 (Too Many Requests).

4. Kết dính các Endpoint (API Glue)

- **Nhiệm vụ:** Dev 5 là người đứng tên cho file main.py nơi khai báo các endpoint FastAPI. Phải đảm bảo các schema request/response (từ file contracts/plan-request.schema.json và contracts/reroute-request.schema.json) được map chuẩn xác vào các Pydantic Models (models.py).

Chỉ đạo nghiệm thu (Acceptance Criteria) cho Dev 5:

Tạo một chuyến đi qua API. Mở Docker, tắt nóng container `itinerary-api` và bật lại (mô phỏng server sập). Gọi API `/reroute`. Nếu chuyến đi cũ vẫn còn nguyên vẹn và hệ thống định tuyến lại thành công, bạn qua ải. Nếu API báo lỗi "Trip not found", bạn phải viết lại toàn bộ code Redis. Làm ngay đi.

Nghe này. Trong toàn bộ kiến trúc hệ thống hiện tại, **Dev 1 (Lead Data)** đang nắm giữ sinh mệnh của toàn bộ dự án MVP này. Đội AI có thể vẽ vời rất hay về OR-Tools hay thuật toán định tuyến, nhưng nguyên lý cơ bản nhất (first principles) ở đây là: "Đưa rác vào $\rightarrow$ ra rác" (Garbage in, garbage out).

Hệ thống hiện tại đang sống ảo trên một cái file `sample_pois.json`. Nếu chúng ta ném nguyên 1000 điểm JSON này vào OR-Tools, hệ thống sẽ sập ngay lập tức vì bùng nổ tổ hợp (Combinatorial Explosion).

Để biến cái này thành một sản phẩm thực tế, đây là cách Dev 1 phải làm việc một cách cực kỳ hardcore. Không lươn lẹo, không đi đường tắt:

1. Xóa bỏ rác (Delete the fake parts)

Phần tốt nhất của một hệ thống là không có hệ thống nào cả, nhưng ở đây, phần tốt nhất của code là code đã bị xóa.

- **Mục tiêu:** Dẹp bỏ ngay cái file `sample_pois.json` đang được mock trong hệ thống.
- **Hành động:** Đi thẳng vào file `backend/app/repository.py` và vứt bỏ đoạn code đọc dữ liệu tĩnh. Chúng ta cần I/O thực sự, không phải đồ giả.

2. Thu thập dữ liệu (The Data Crawl)

Thuật toán không thể tự tưởng tượng ra thành phố Huế. Nó cần tọa độ.

- **Nhiệm vụ:** Viết một script để tự động crawl khoảng 500 điểm đến (POIs - Points of Interest) tại Huế từ OpenStreetMap (OSM) hoặc Google Places.
- **Dữ liệu bắt buộc phải có:** Tọa độ (Lat/Long), Nhãn phân loại (Soft tags như "culture", "spicy", "outdoor"), Chi phí (Budget), và đặc biệt là Giờ mở/đóng cửa.

3. Làm sạch dữ liệu bằng tay (The Hardcore Manual Work)

Đây là phần đau đớn nhất nhưng không thể dùng AI để tự động hóa lúc này.

- **Rủi ro chí mạng (Critical Risk):** Nếu API crawl về báo sai giờ đóng cửa, cái lõi OR-Tools của chúng ta sẽ định tuyến sai và dẫn khách du lịch đến đập mặt vào một cái cửa đã khóa kính.
- **Cách làm:** Dev 1 bắt buộc phải review và làm sạch 500 điểm dữ liệu này hoàn toàn bằng tay. Phải gọi điện, phải check Google Maps, phải làm mọi cách để đảm bảo biến số Time Windows (Giờ hoạt động) là chính xác 100%.

4. Đưa vào cơ sở dữ liệu không gian (Deploy to PostGIS)

Khi dữ liệu đã sạch, chúng ta cần một động cơ truy vấn đủ mạnh để lọc 1000 điểm xuống còn 50 điểm cực nhanh trước khi đẩy cho thuật toán.

- **Công cụ:** Sử dụng file script `scripts/seed_pois.py` đã có sẵn.
- **Cách làm:** Convert tập dữ liệu POI vừa làm sạch thành định dạng SQL seed, sau đó import thẳng vào container PostGIS đang chạy trong `docker-compose.yml`.
- **Kết nối:** Phải đảm bảo database này có thể thực thi các truy vấn không gian (Spatial query) thực sự, thay thế hoàn toàn cho cái ảo giác (illusion) JSON hiện tại.

Tóm lại: Việc của Dev 1 không phải là ngồi code những thứ màu mè. Việc của Dev 1 là xây dựng một nền móng vật lý chính xác tuyệt đối (tọa độ không gian và thời gian) cho thành phố Huế. Hãy làm cho xong việc này ngay lập tức.

Yêu cầu Nhìn từ góc độ nguyên lý cơ bản (First Principles) của bài toán tối ưu hóa, hệ thống cơ sở dữ liệu không gian (PostGIS) không chỉ dùng để lưu trữ tọa độ. Nó phải đóng vai trò là một "bộ lọc vật lý" để loại bỏ các điểm đến phi logic trước khi ném mảng dữ liệu vào bộ não OR-Tools.

Nếu dữ liệu đầu vào thiếu chi tiết, hệ thống định tuyến sẽ bị "ảo giác" ngay trong thế giới thực. Dưới đây là phân tích chi tiết.

1. Yêu cầu Schema dữ liệu đầu vào bắt buộc cho PostGIS

Để thỏa mãn cả 4 lớp kiến trúc (đặc biệt là Lớp Tiền xử lý và Lớp Lõi Toán học), mỗi điểm POI (Point of Interest) khi nạp vào PostGIS qua script `seed_pois.py` bắt buộc phải có các trường dữ liệu (attributes) sau:

- **Ràng buộc Không gian (Spatial Constraints):**
  - `geom`: Dữ liệu tọa độ (Latitude, Longitude) được ép kiểu thành `geometry(Point, 4326)` trong PostGIS để hỗ trợ truy vấn bán kính cực nhanh.
- **Ràng buộc Thời gian (Hard Time Constraints) - Tối quan trọng:**
  - `open_time` & `close_time`: Giờ mở cửa và đóng cửa. (VD: 08:00 - 17:00). Đây là biến số sống còn cho bài toán VRPTW (Vehicle Routing Problem with Time Windows) của OR-Tools.
  - `typical_duration`: Thời gian tham quan trung bình (VD: 45 phút, 120 phút). Thuật toán cần số này để tính toán xem khách đi xong điểm A có kịp giờ đến điểm B không.
- **Ràng buộc Sở thích & Thuộc tính (Soft Constraints & Context):**
  - `tags`: Mảng các từ khóa (VD: `["culture", "spicy", "heritage"]`). Bắt buộc phải có để map với mảng `soft_tags` mà AI Gateway (LLM) trả về.
  - `is_outdoor`: Cờ boolean (True/False). Cực kỳ quan trọng để kích hoạt cơ chế Fallback thời tiết. Nếu trời mưa, hệ thống tự động gán `weather_penalty` rất cao cho các điểm `is_outdoor = True` để OR-Tools né tránh.
  - `estimated_budget`: Chi phí dự kiến cho một người, dùng để lọc qua SQL theo yêu cầu `budget_max` của user.

2. Dùng mỗi OpenStreetMap (OSM) có đủ không?

Lý do: Tài liệu thiết kế đã lường trước điều này và yêu cầu rõ ở Task 1.2 là "Crawl & Clean 500-1000 POI Huế (OSM + Google Places)".

1. **Điểm mạnh của OSM:** Rất tốt để lấy tọa độ (Geom) chính xác, vẽ polygon của các khu vực lớn (như Đại Nội Huế), và hoàn toàn miễn phí.
2. **Lỗ hổng chí mạng của OSM:** OSM được đóng góp bởi cộng đồng, do đó dữ liệu về **Giờ đóng/mở cửa** và **Chi phí (Budget)** cực kỳ nghèo nàn và thiếu cập nhật. Bản thiết kế đã cảnh báo rủi ro ở mức đỏ (CRITICAL RISK): *Nếu dữ liệu crawl sai giờ đóng cửa, OR-Tools sẽ đưa khách đến cái cửa đóng kín*.

3. Đề xuất các trang cào dữ liệu (Crawl Sources) để có Data "Sạch và Chuẩn nhất"

Để Dev 1 có thể hoàn thành xuất sắc 500 điểm POI tại Huế với độ tin cậy cao nhất, chiến lược cào dữ liệu phải là **Cross-validation (Đối chiếu chéo)** từ các nguồn sau:

- **Nguồn 1: Google Places API (Nguồn chân lý về Thời gian & Tọa độ)**
  - *Mục đích:* Lấy tọa độ chính xác, `open_time`, `close_time`, và quan trọng nhất là dữ liệu "Popular Times" (thời gian đông khách/vắng khách) để tối ưu lịch trình.
  - *Hành động:* Quét qua Google Places. Đây là nguồn chuẩn nhất về giờ hoạt động của các địa điểm du lịch, bảo tàng, và lăng tẩm tại Huế.
- **Nguồn 2: TripAdvisor (Nguồn chân lý về Soft Tags & Budget)**
  - *Mục đích:* Khai thác các hệ thống phân loại (`tags`) và phân khúc giá (`estimated_budget`).
  - *Hành động:* Cào các review và tag để xác định xem quán bún bò này thuộc tag "spicy", "local food", hay lăng tẩm kia thuộc tag "culture", "history". TripAdvisor phân loại outdoor/indoor cũng rất rõ ràng.
- **Nguồn 3: ShopeeFood / Foody / Capichi (Riêng cho F&B)**
  - *Mục đích:* Độ chính xác tuyệt đối cho mảng Ẩm thực (Quán ăn, Cafe).
  - *Hành động:* Huế nổi tiếng về ẩm thực đường phố và quán chè, bún bò mở theo giờ rất ngặt nghèo (VD: chỉ bán từ 14:00 - 18:00). Google Maps đôi khi không cập nhật nhanh bằng các app giao đồ ăn nội địa. Cào ShopeeFood sẽ lấy được giờ bán thực tế và mức giá chính xác nhất.
- **Nguồn 4: Cổng thông tin du lịch Thừa Thiên Huế (Data Chính phủ)**
  - *Mục đích:* Chốt giờ đóng/mở cửa của các di tích thuộc Trung tâm Bảo tồn Di tích Cố đô Huế.
  - *Hành động:* Giờ đóng mở cửa của Đại Nội, Lăng Tự Đức, Khải Định thay đổi theo mùa (Mùa hè khác mùa đông). Lấy số liệu trực tiếp từ website nhà nước là an toàn nhất.

Chỉ đạo thực thi cho Dev 1: Đừng viết một cái script cào mù quáng. Hãy viết một pipeline: Dùng thuật toán lấy danh sách tọa độ gốc từ OSM $\rightarrow$ Đẩy tên các điểm đó qua Google Places để lấy giờ mở cửa $\rightarrow$ Đẩy qua TripAdvisor lấy Tag $\rightarrow$ Đổ tất cả ra một file CSV $\rightarrow$ **Dùng mắt người (Manual Review) dò lại 500 điểm này** trước khi seed vào PostGIS. Làm kỹ bước này, phần toán học phía sau sẽ chạy mượt mà.

Tiếp tục với lăng kính nguyên lý cơ bản (First Principles): Nếu Dev 5 xây dựng "Trí nhớ" (Redis State), thì **Dev 6 (Security / Event & Notification)** chính là người xây dựng **Hệ Thần Kinh Phản Xạ (The Reflex System)** cho toàn bộ kiến trúc động này.

Bài toán cốt lõi của một hệ thống định tuyến động (Dynamic Routing) không nằm ở việc tính đường một lần, mà là ở khả năng **chữa cháy (re-route)** khi thực tế chệch hướng khỏi kế hoạch.

Theo phân công nhiệm vụ, Hard Requirement của Dev 6 là: *"Bắt event từ Mobile (/reroute), tính toán delay và đẩy Notification (FCM) về lại máy."*. Dưới đây là mổ xẻ chi tiết các vật phẩm bàn giao (Deliverables) không được phép sai sót của Dev 6:

1. Xây dựng Bộ tính toán Độ trễ (The Delay Engine)

- **[FACT] Tình trạng:** Hiện tại app Mobile đang dùng nút bấm tay "I'm delayed" (Mockup).
- **Hành động bắt buộc:** Dev 6 phải viết logic xử lý cho API `POST /reroute`. Khi Mobile bắn một tín hiệu chứa tọa độ và thời gian hiện tại (thông qua file schema `contracts/reroute-request.schema.json`), Dev 6 phải:
    1. Truy xuất State chuyến đi từ Redis (thông qua hàm của Dev 5).
    2. Lấy giờ dự kiến (ETA) của điểm đến hiện tại đem trừ đi giờ thực tế user đang đứng.
    3. Tính ra biến số `delay_minutes` chính xác tuyệt đối.

2. Định tuyến lại (Trigger Core) & Bảo mật (Security/Auth)

- **[CRITICAL RISK]:** Kẻ tấn công có thể spam hàng ngàn request giả mạo vào cổng `/reroute` làm cháy server OSRM và API nội bộ.
- **Hành động bắt buộc:** Với vai trò là người nắm Security, Dev 6 phải bọc API `/reroute` bằng một lớp xác thực (Authentication). Tối thiểu là kiểm tra Trip ID / Device Token hợp lệ.
- Sau khi xác thực và tính ra `delay`, nếu độ trễ vượt ngưỡng cho phép (VD: > 15 phút), Dev 6 mới được phép "đánh thức" Dev 4 (Maths) để chạy lại OR-Tools.

3. Bắn Firebase Cloud Messaging (The Wake-up Call)

- **[INFERENCE] Phân tích kiến trúc:** Tại sao lại cần FCM (Firebase Cloud Messaging)? Vì điện thoại (iOS/Android) có cơ chế quản lý pin cực kỳ khắt khe. Khi user đút điện thoại vào túi, App sẽ bị OS đóng băng (Suspended). Khi server tính xong đường mới, Server không thể tự nhiên "gọi" vào một cái App đang ngủ nếu không có FCM.
- **Hành động bắt buộc:** * Dev 6 phải cấu hình Firebase Project và tích hợp SDK Admin vào backend.
    - Khi OR-Tools tính xong mảng 10 điểm hành trình mới, Dev 6 phải đóng gói mảng đó thành một Data Payload (chứ không chỉ là Notification text thuần túy).
    - Bắn cục Payload này qua FCM về đúng cái Device Token đã kích hoạt sự kiện. Hành động này sẽ ép hệ điều hành điện thoại bừng tỉnh, cập nhật lại

bản đồ mà người dùng không cần phải mở khóa màn hình để "kéo thả tải lại" (pull-to-refresh).

Chỉ đạo nghiệm thu (Acceptance Criteria) cho Dev 6:

Bạn làm việc độc lập nhưng kết quả phải ghép nối hoàn hảo. Cách nghiệm thu: Dùng Postman bắn một lệnh giả lập user đi trễ 45 phút vào cổng `/reroute`. Sau đó, ngồi nhìn màn hình điện thoại (đang khóa màn hình). Trong vòng dưới 3 giây, màn hình phải sáng lên, đẩy ra một thông báo Push Notification (FCM): *"Lịch trình đã được tính lại do trễ giờ"*, đi kèm với lộ trình mới được cập nhật ngầm. Không làm được điều này, hệ thống chỉ là một cái web-view phế vật.